```
NEURAL
WINGS
[ Server V1.0 ]
```

LANGUAGE C++17 | TRANSPORT UDP + WEBRTC | BUILD CMAKE + VCPKG | ROLE AUTHORITATIVE SERVER

Neural Wings Server 是 Neural_Wings-demo 的配套权威服务器工程，负责多人会话接入、玩家身份分配、飞行状态广播、对象回收同步、聊天与昵称管理。项目基于 **nbnet**，统一抽象了 **UDP + WebRTC DataChannel** 传输层，使桌面端与 Web 端都能接入同一套服务端逻辑。

🎯 1. 项目定位与目标

该仓库是一个“**服务器权威（Authoritative）**”网络后端，目标是保证多人状态一致与通信可靠。

核心职责：

- **连接接入与会话管理**：处理新连接、断线、超时移除。
- **身份与对象管理**：为玩家分配 `ClientID`，维护 `ClientID ↔ ConnectionHandle ↔ UUID` 映射。
- **实时状态广播**：收集客户端位置状态，以固定 tick 广播给所有在线玩家。
- **对象生命周期同步**：接收 `ObjectRelease`，广播 `ObjectDespawn`，避免残留幽灵对象。
- **聊天系统与昵称系统**：支持公聊、私聊模式切换、昵称冲突校验、在线玩家元信息广播。
- **跨端协议统一**：依赖 `shared/Engine/Network` 与客户端共享协议定义，确保字节级兼容。

🏗 2. 系统架构总览

2.1 模块分层

- **入口层 (`main.cpp`)**：参数解析、信号处理、固定帧 tick 循环。
- **生命周期层 (`Lifecycle.cpp`)**：启动/停止 nbnet、注册传输驱动、轮询事件。
- **连接分发层 (`Connection.cpp`)**：处理连接事件、消息分发、客户端移除与超时回收。
- **同步层 (`StateSync.cpp`)**：欢迎包、玩家元数据同步、位置广播、可靠/不可靠通道映射。

- **业务层 (`Chat.cpp`)**：聊天指令解析、节流、防冲突昵称、系统消息反馈。
- **协议层 (`shared/Engine/Network`)**：消息枚举、POD 结构、序列化/反序列化工具。

2.2 权威设计原则

- **客户端上报，服务端裁决并再分发**：客户端只上报自己的状态，不直接写入他人状态。
- **关键事件走可靠通道**：欢迎包、聊天、昵称、玩家元数据、对象销毁使用可靠传输。
- **高频状态走不可靠通道**：位置广播使用不可靠传输，优先低延迟。
- **状态与连接解耦：**`ObjectRelease` 允许“释放游戏对象但保留连接”，支持玩家停留在菜单界面。
- **持久身份识别**：使用 `NetUUID` 识别回归玩家并复用历史 `ClientID`。

🔄 3. 服务端生命周期与主循环

3.1 启动阶段

`GameServer::Start()` 的关键动作：

1. 首次进程启动时注册 nbnet 驱动（避免重复注册断言）。
2. 始终注册 UDP 驱动。
3. 若定义 `NW_ENABLE_WEBRTC_C`，同时注册 WebRTC C 驱动。
4. 调用 `NBN_GameServer_StartEx("neural_wings", port, false)` 监听端口。
5. 初始化运行标记与 tick 计数。

3.2 运行阶段

`main.cpp` 中使用约 `33ms` 间隔执行一次 `Tick()`：

- `NBN_GameServer_Poll()` 持续拉取事件。
- 事件分发：`NEW_CONNECTION` / `CLIENT_DISCONNECTED` / `CLIENT_MESSAGE_RECEIVED`。
- `RemoveTimedOutClients()` 做应用层超时清理（默认 5 秒）。
- `BroadcastPositions()` 广播所有已上报玩家状态。
- `NBN_GameServer_SendPackets()` 统一刷新发送队列。

3.3 停止阶段

- 响应 Ctrl+C / SIGINT / SIGTERM。
- 调用 `NBN_GameServer_Stop()`。
- 清理 `m_clients`、`m_connIndex`、`m_nicknameIndex` 等索引。

📡 4. 网络协议与消息流

4.1 共享协议文件

所有消息定义位于：

- `shared/Engine/Network/NetTypes.h`
- `shared/Engine/Network/Protocol/MessageTypes.h`
- `shared/Engine/Network/Protocol/Messages.h`
- `shared/Engine/Network/Protocol/PacketSerializer.h`

这些文件必须与客户端仓库保持同步。

4.2 消息类型分组

- **连接类**：`ClientHello` / `ServerWelcome` / `Heartbeat` / `ClientDisconnect`
- **状态同步类**：`PositionUpdate` / `PositionBroadcast` / `ObjectRelease` / `ObjectDespawn`
- **聊天元数据类**：`ChatRequest` / `ChatBroadcast` / `NicknameUpdateRequest` / `NicknameUpdateResult` / `PlayerMetaSnapshot` / `PlayerMetaUpsert` / `PlayerMetaRemove`

4.3 典型消息时序

新玩家接入

1. 客户端连接后发送 `ClientHello(uuid)`。
2. 服务端检查 UUID：
   - 已知且离线：复用旧 `ClientID`。
   - 已知且在线：拒绝重复会话。
   - 新 UUID：注册新身份。
3. 服务端回发 `ServerWelcome`。
4. 服务端回发 `NicknameUpdateResult(Accepted)`（含当前权威昵称）。
5. 服务端回发 `PlayerMetaSnapshot`（在线玩家快照）。
6. 服务端广播 `PlayerMetaUpsert`（通知其他玩家该玩家在线）。

对象释放

1. 客户端发送 `ObjectRelease(clientID, objectID)`。
2. 服务端向其他玩家广播 `ObjectDespawn(ownerID, objectID)`。
3. 服务端清除该玩家对象状态，但保留连接。
4. 进入短暂 `release fence` 窗口，忽略迟到的旧位置包。

🧱 5. 状态同步机制

5.1 关键数据结构

`GameServer::ClientState` 包含：

- 连接标识：`connHandle`
- 身份标识：`id`, `uuid`
- 对象状态：`objectID`, `lastTransform`, `hasTransform`
- 玩家元数据：`nickname`, 私聊目标
- 时序控制：`lastSeen`, `releaseFenceUntil`, `lastChatTime`

5.2 索引设计

- `m_clients`: `ClientID -> ClientState`
- `m_connIndex`: `ConnectionHandle -> ClientID`
- `m_uuidIndex`: `UUID -> ClientID`
- `m_nicknameIndex`: `normalized nickname -> ClientID`

该多索引结构确保事件分发、身份复用、昵称冲突检测均为 O(1) 近似查找。

5.3 通道策略

`StateSync.cpp` 使用通道约定：

- `channel = 0` -> nbnet 可靠通道
- `channel = 1` -> nbnet 不可靠通道

当前策略：

- **可靠**：欢迎包、对象销毁、聊天、昵称更新、玩家元数据。
- **不可靠**：`PositionBroadcast`（高频状态）。

5.4 超时回收

若 `now - lastSeen > 5000ms`，服务端主动移除客户端并尝试关闭底层传输，避免“僵尸连接”。

💬 6. 聊天与昵称子系统

6.1 昵称规则

- 长度：`3 ~ 16`
- 字符：仅允许字母、数字、下划线
- 比较：大小写不敏感（归一化后判重）
- 冲突：若被占用，返回 `NicknameUpdateStatus::Conflict`

6.2 聊天安全与风控

- 文本长度上限：`256`
- 发送节流：`300ms`（每客户端）
- 非法请求：直接拒绝或系统提示
- 客户端不可发送系统消息类型（防伪造）

6.3 指令模式

- `/help`：帮助信息
- `/w <nickname>`：进入私聊模式
- `/a`：切回公聊模式

私聊模式下目标掉线会自动回退公聊并提示，避免消息黑洞。

🛠 7. 构建与运行（Windows / Linux）

7.1 先决条件

- CMake `>= 3.11`
- C++17 编译器（MSVC/GCC/Clang）
- `vcpkg`（依赖：`libdatachannel`, `openssl`）

7.2 Windows 构建

```
# 1) 获取 vcpkg（若尚未存在）
git clone https://github.com/microsoft/vcpkg.git
.\vcpkg\bootstrap-vcpkg.bat

# 2) 配置
cmake -B build -S . `
  -DCMAKE_TOOLCHAIN_FILE=vcpkg/scripts/buildsystems/vcpkg.cmake `
  -DVCPKG_TARGET_TRIPLET=x64-windows

# 3) 编译
cmake --build build --config Debug

# 4) 运行（默认端口 7777）
.\build\Debug\Neural_Wings-server.exe

# 5) 指定端口
.\build\Debug\Neural_Wings-server.exe 9000
```

7.3 Linux 构建

```
./vcpkg/bootstrap-vcpkg.sh
./vcpkg/vcpkg install libdatachannel:x64-linux

cmake -B build_wsl -S . \
  -DCMAKE_TOOLCHAIN_FILE=vcpkg/scripts/buildsystems/vcpkg.cmake \
  -DCMAKE_BUILD_TYPE=Debug -G Ninja

cmake --build build_wsl -j"$(nproc)"
./build_wsl/Neural_Wings-server
```

7.4 VS Code 预置任务

`.vscode/tasks.json` 已提供：

- `Build Debug / Build Release`
- `WSL Install Deps`
- `WSL Build Debug`
- `WSL Run Server`

`.vscode/launch.json` 已提供 Windows 与 WSL 的调试启动配置，可直接使用。

---

8. 详细文件层级结构

```
Neural_Wings-server/
├── .vscode/
│   ├── launch.json                     # Windows/WSL 调试配置
│   └── tasks.json                      # 构建、运行、清理任务
│
├── src/                                # ================= 服务器核心实现
=================
│   ├── main.cpp                        # 程序入口、参数解析、30Hz 主循环、信号处理
│   ├── GameServer.h                    # 服务器总类声明、状态结构、核心接口
│   ├── Lifecycle.cpp                   # Start/Stop/Tick 生命周期与 nbnet 驱动注册
│   ├── Connection.cpp                  # 连接事件处理、消息分发、超时与断线回收
│   ├── StateSync.cpp                   # 欢迎包、对象销毁、元数据与位置广播
│   ├── Chat.cpp                        # 聊天、私聊模式、昵称校验与系统消息
│   └── nbnet_server_impl.c             # nbnet 实现编译单元（C 编译，含驱动实现）
│
├── shared/Engine/Network/              # ===== 与客户端共享协议（必须同步） =====
│   ├── NetTypes.h                      # ID/UUID/默认端口等基础网络类型
│   └── Protocol/
│       ├── MessageTypes.h              # NetMessageType 枚举定义
│       ├── Messages.h                  # 所有打包消息 POD 结构
│       └── PacketSerializer.h          # 读写序列化工具（header-only）
│
├── third_party/
│   └── nbnet/                          # nbnet 及 UDP/WebRTC 驱动源码
│
├── CMakeLists.txt                      # 构建入口，依赖查找与平台链接
├── vcpkg.json                          # 依赖清单（libdatachannel, openssl）
└── README.md
```